Restaurant Rush

SC2002 Object-Oriented Design & Programming - Group Assignment Report
Lab group: [group] | Members: [names]
Repository: https://github.com/manus369/sc2002-lab-poject (tags `stage-1`, `stage-2`, `stage-3`)

1. Overview

Restaurant Rush is a turn-based command-line game in Java. The player is the manager of a restaurant with 2 tables and one Waiter, Chef and Cashier (and a Host in Stage 3). Six customers arrive on turns 1, 4, 7, 10, 13 and 16. Each turn the manager chooses one task for the Waiter (seat, take an order or serve) and which order the Chef cooks. The Cashier, eating, satisfaction loss and walk-outs are automatic. After turn 18 the game is a victory if at least 4 customers have paid, revenue is at least $45 and the average satisfaction of paid customers is at least 60.

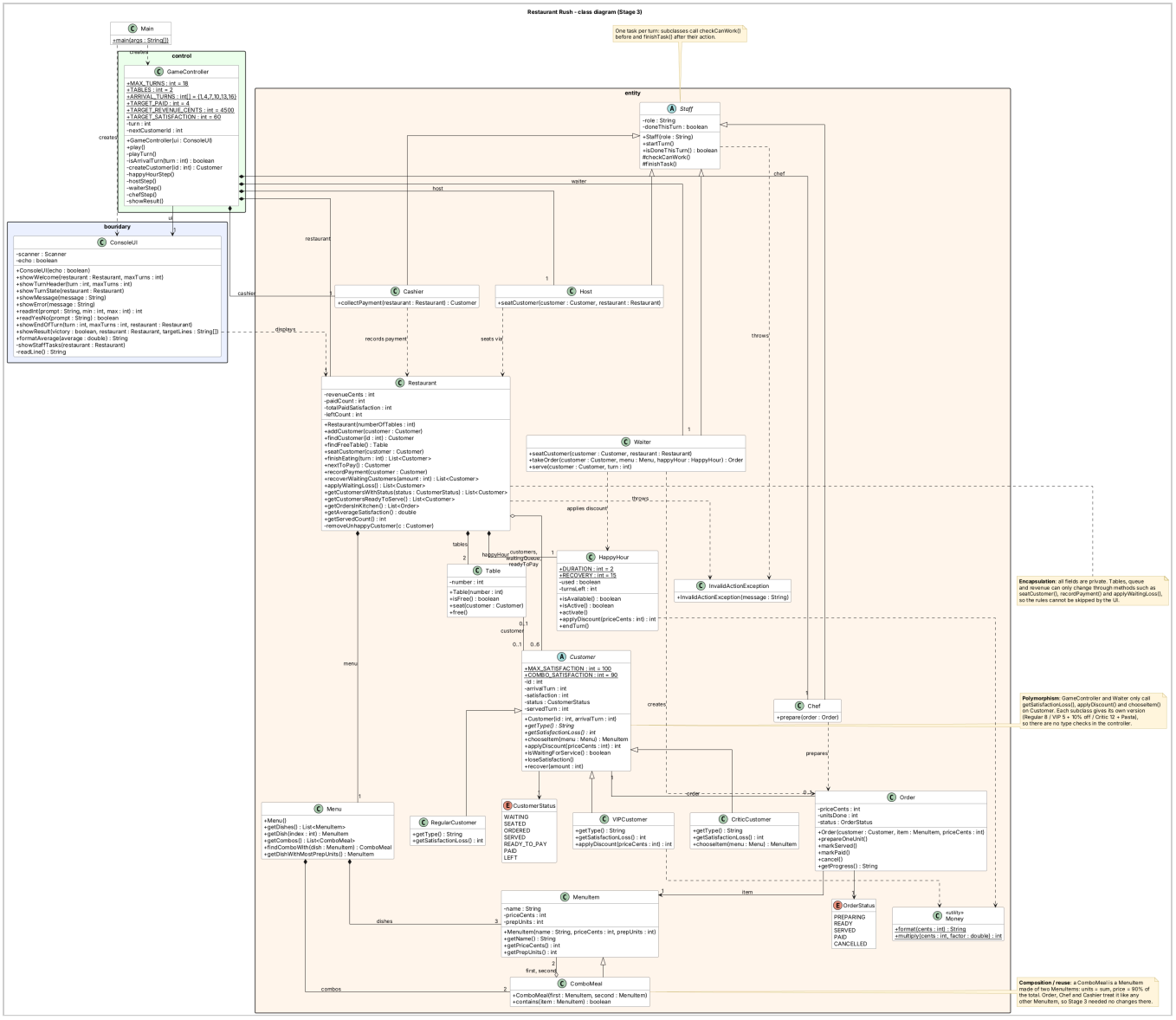
The code uses the entity-control-boundary structure taught in the course: `rush.entity` holds the game objects and rules, `rush.control.GameController` runs the turn steps, and `rush.boundary.ConsoleUI` only prints the state and reads input.

2. Rules we defined

Rule	Our choice
Food choice	Customer *n*'s usual dish is menu dish (*n*-1), wrapping around: Salad, Burger, Pasta, Salad, ...
Critic behaviour	Loses 12 per waiting turn and always orders the dish with the most preparation units (Pasta), never a combo.
Combo choice (Stage 3)	A customer with satisfaction 90 or more when the order is taken upgrades to the first combo containing their usual dish. Combos: Salad + Burger ($21.60, 2 units), Salad + Pasta ($24.30, 3 units).
Arrival line-up (Stage 2+)	C1 Regular, C2 Critic, C3 VIP, C4-C6 Regular.
Rounding	Money is stored in integer cents. Each discount is rounded to the nearest cent: VIP first, then Happy Hour.
Turn order	Arrivals, display, Happy Hour, Host, Waiter, Chef, Cashier, eating, satisfaction loss. The Cashier runs before eating, so a customer who becomes READY_TO_PAY pays next turn at the earliest.

3. UML class diagram

The diagram below matches the `stage-3` code. Simple getters and setters are left out. The notes show where encapsulation, polymorphism and composition are used.

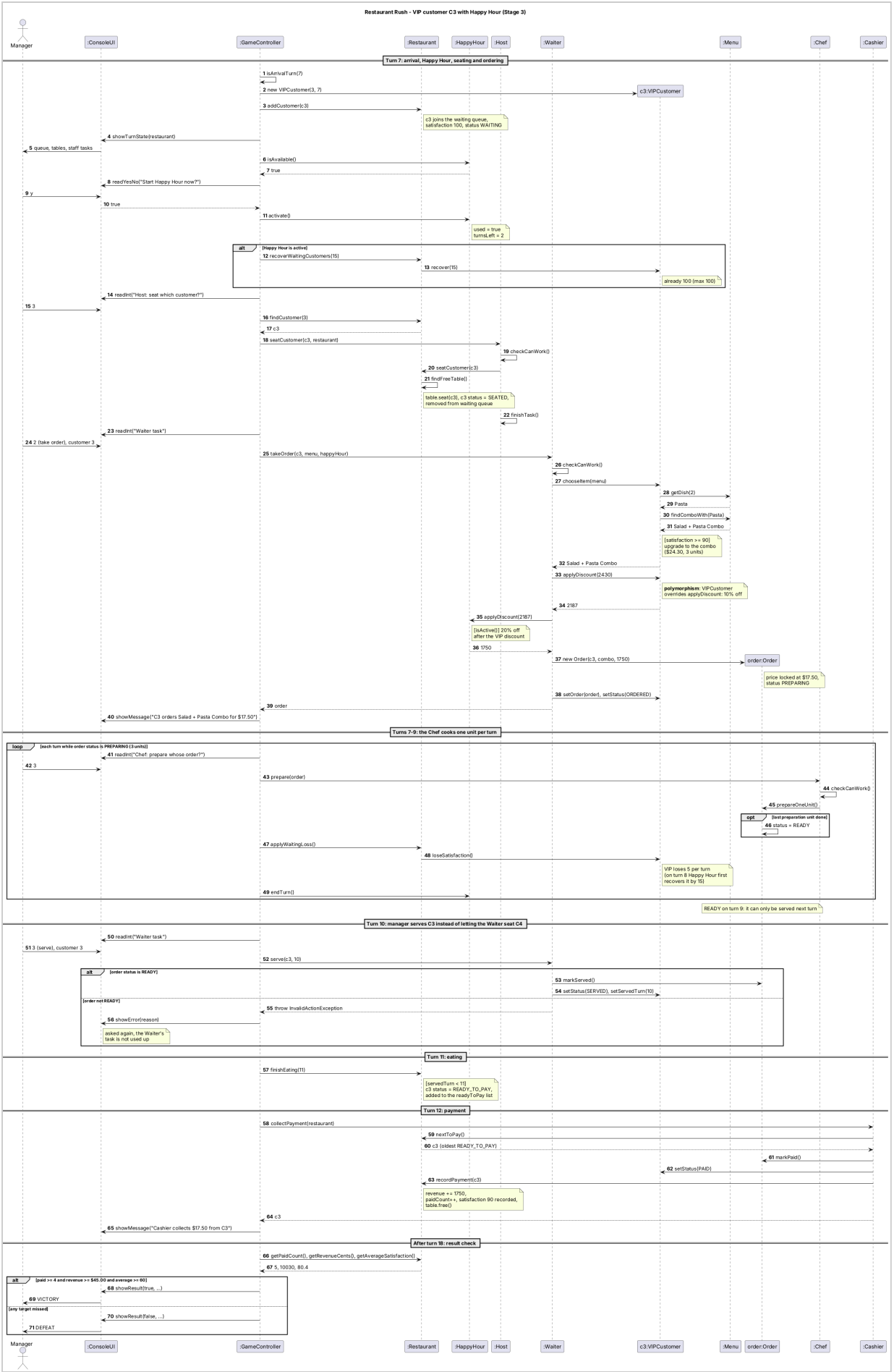


Notes on the diagram

- **Inheritance:** `Customer` (abstract) has three subclasses; `Staff` (abstract) has Host, Waiter, Chef and Cashier; `ComboMeal` extends `MenuItem`.
- **Composition:** the `Restaurant` creates and owns its 2 `Table`s, its `Menu` and its `HappyHour`; the `Menu` owns 3 dishes and 2 combos; the `GameController` owns the restaurant and the staff.
- **Aggregation:** a `ComboMeal` refers to two existing `MenuItems` that also exist on their own.
- **Associations:** a Customer has 0..1 Order and sits at 0..1 Table; an Order refers to exactly one MenuItem.
- **Enums:** `CustomerStatus` and `OrderStatus` describe where a customer and an order are in their life cycle.

4. UML sequence diagram

Scenario (from `runs/stage3-victory.txt`): the VIP C3 arrives on turn 7 and the manager chooses to start Happy Hour. The Host seats C3 and the Waiter takes the order. C3 upgrades to a combo, and the price is worked out by `VIPCustomer.applyDiscount` (10% off) and then `HappyHour.applyDiscount` (20% off): $24.30 → $21.87 → $17.50. The Chef cooks one unit per turn (loop) until the order is READY, the Waiter serves it on the next turn, C3 eats, the Cashier collects payment and finally the result is checked. The manager's choice to start Happy Hour changes the bill (from $21.87 to $17.50) and the satisfaction recorded at payment (+15 on turn 8).


Restaurant Rush – VIP customer C3 with Happy Hour (Stage 3)
Manager
:ConsoleUI
:GameController
:Restaurant
:HappyHour
:Host
:Waiter
c3:VIPCustomer
:Menu
order:Order
:Chef
:Cashier
Turn 7: arrival, Happy Hour, seating and ordering
1 isArrivalTurn(7)
2 new VIPCustomer(3, 7)
3 addCustomer(c3)
c3 joins the waiting queue, satisfaction 100, status WAITING
4 showTurnState(restaurant)
5 queue, tables, staff tasks
6 isAvailable()
7 true
8 readYesNo("Start Happy Hour now?")
9 y
10 true
11 activate()
used = true turnsLeft = 2
alt
[Happy Hour is active]
12 recoverWaitingCustomers(15)
13 recover(15)
already 100 (max 100)
14 readInt("Host: seat which customer?")
15 3
16 findCustomer(3)
17 c3
18 seatCustomer(c3, restaurant)
19 checkCanWork()
20 seatCustomer(c3)
21 findFreeTable()
table.seat(c3), c3 status = SEATED, removed from waiting queue
22 finishTask()
23 readInt("Waiter task")
24 2 (take order), customer 3
25 takeOrder(c3, menu, happyHour)
26 checkCanWork()
27 chooseItem(menu)
28 getDish(2)
29 Pasta
30 findComboWith(Pasta)
31 Salad + Pasta Combo
[satisfaction >= 90] upgrade to the combo ($24.30, 3 units)
32 Salad + Pasta Combo
33 applyDiscount(2430)
polymorphism: VIPCustomer overrides applyDiscount: 10% off
34 2187
35 applyDiscount(2187)
[isActive()] 20% off after the VIP discount
36 1750
37 new Order(c3, combo, 1750)
price locked at $17.50, status PREPARING
38 setOrder(order), setStatus(ORDERED)
39 order
40 showMessage("C3 orders Salad + Pasta Combo for $17.50")
Turns 7–9: the Chef cooks one unit per turn
loop
[each turn while order status is PREPARING (3 units)]
41 readInt("Chef: prepare whose order?")
42 3
43 prepare(order)
44 checkCanWork()
45 prepareOneUnit()
opt
[last preparation unit done]
46 status = READY
47 applyWaitingLoss()
48 loseSatisfaction()
VIP loses 5 per turn (on turn 8 Happy Hour first recovers it by 15)
49 endTurn()
READY on turn 9: it can only be served next turn
Turn 10: manager serves C3 instead of letting the Waiter seat C4
50 readInt("Waiter task")
51 3 (serve), customer 3
52 serve(c3, 10)
alt
[order status is READY]
53 markServed()
54 setStatus(SERVED), setServedTurn(10)
[order not READY]
55 throw InvalidActionException
56 showError(reason)
asked again, the Waiter's task is not used up
Turn 11: eating
57 finishEating(11)
[servedTurn < 11] c3 status = READY_TO_PAY, added to the readyToPay list
Turn 12: payment
58 collectPayment(restaurant)
59 nextToPay()
60 c3 (oldest READY_TO_PAY)
61 markPaid()
62 setStatus(PAID)
63 recordPayment(c3)
revenue += 1750, paidCount++, satisfaction 90 recorded, table.free()
64 c3
65 showMessage("Cashier collects $17.50 from C3")
After turn 18: result check
66 getPaidCount(), getRevenueCents(), getAverageSatisfaction()
67 5, 10030, 80.4
alt
[paid >= 4 and revenue >= $45.00 and average >= 60]
68 showResult(true, ...)
69 VICTORY
[any target missed]
70 showResult(false, ...)
71 DEFEAT

5. Design considerations

5.1 Key abstractions and responsibilities

Class	Responsibility
GameController	Runs the 18 turns in the fixed step order, asks the UI for the manager's choices, and checks the victory targets.
ConsoleUI	Prints the state and reads input. Rejects input that is not a number in range. Holds no game rules.
Restaurant	Holds the tables, queue and customers; seats customers; handles eating, payment records, satisfaction loss and walk-outs; keeps the totals.
Customer (+ subclasses)	Satisfaction and status of one customer. Decides what to order, how fast satisfaction drops and the bill discount.
Order	Locked price, cooking progress and status of one order.
Staff (+ Host, Waiter, Chef, Cashier)	Each staff member does one task per turn and checks that the task is allowed, throwing `InvalidActionException` with a reason if not.
Menu, MenuItem, ComboMeal	The dishes and combos with price (cents) and preparation units.
HappyHour	Once-per-game activation, turns left, the 20% discount and the recovery amount.

5.2 Design principles in our code

- **Single Responsibility:** input/output is only in `ConsoleUI`; cooking progress is only in `Order`; payments only in `Cashier` and `Restaurant.recordPayment`. When we changed the screen layout no rule class changed.
- **Open-Closed:** adding VIP and Critic in Stage 2 meant two new subclasses and one line each in `createCustomer`; the turn steps were not edited. ComboMeal was added in Stage 3 without changing `Order`, `Chef` or `Cashier`.
- **Liskov Substitution:** any `Customer` subclass can be used wherever a `Customer` is expected, and a `ComboMeal` wherever a `MenuItem` is expected; they keep the same meaning (e.g. a discount never makes the price negative).
- **Interface Segregation and Dependency Inversion:** we did not use these. The brief says not to add interfaces just for a checklist, and with one UI and one kind of each object an interface would add files without benefit. See the trade-off below.

5.3 Trade-offs

- `GameController` uses the concrete `ConsoleUI` class. This is simpler, but we cannot swap in a GUI or a fake UI for tests without changing the controller. Instead we test with input scripts piped into the real game (`runs/*.txt`).
- `Restaurant` has quite a few methods. We kept seating, eating and payment records together because they all change the same tables and lists; splitting them would mean passing those lists around.

- Waiting rules use the `CustomerStatus` enum instead of a class per state. This keeps the number of classes small; the cost is that status checks are spread over a few methods.
- `Waiter.takeOrder` gained a `HappyHour` parameter in Stage 3. This is a small change to an earlier class but keeps the price calculation in one place.

5.4 Alternative design we considered

We considered a single `Customer` class with a `CustomerType` enum and `switch` statements for loss rate, discount and order choice. It would have fewer classes, but every new customer type would mean editing every switch, and the brief asks for behaviour to be dispatched through the customer objects. We also considered a `PricingPolicy` interface for Happy Hour; with only one promotion we kept a plain `HappyHour` class.

5.5 How the player's choices change the result

- The Waiter can do only one task per turn. On turn 10 of the Stage 2 and 3 runs the manager must choose between serving a READY order and seating a new arrival; whoever waits loses satisfaction.
- Leaving the Critic in the queue (loses 12 per turn) makes them walk out on turn 12 (Stage 2 and 3 defeat runs).
- Seating and ordering quickly keeps satisfaction at 90+, so customers upgrade to combos and pay more (Stage 3 victory: $100.30).
- Starting Happy Hour when customers are waiting helps satisfaction; starting it on turn 1 with one customer wastes it (Stage 3 defeat).

6. Functional evidence

6.1 Reproducible runs

Each script is replayed with `./run.sh --echo < runs/<script>.txt` on its tag; the `.out` file is the transcript. All six were re-run on their tags and match.

Tag	Script	Result
stage-1	stage1-victory	VICTORY - 5 paid, $66.00, avg 79.2 (includes rejected inputs)
stage-1	stage1-defeat	DEFEAT - 1 paid, $15.00; C1 forgotten and walks out on turn 13
stage-2	stage2-victory	VICTORY - 5 paid, $67.20, avg 76.0
stage-2	stage2-defeat	DEFEAT - 3 paid, $34.20; the Critic walks out
stage-3	stage3-victory	VICTORY - 5 paid, $100.30, avg 80.4
stage-3	stage3-defeat	DEFEAT - 3 paid, $48.15; revenue met but not enough customers paid

6.2 Checks against the brief

Requirement	Where it is shown
Section 3.3 example (Pasta: seated, ordered, READY, served, ready to pay, paid with 76)	C3 in stage1-victory, turns 7-12: satisfaction 92, 84, 76, served at 76, pays $18.00 with 76.
A dish finished this turn is served next turn at the earliest	The Waiter acts before the Chef, so a READY order can only be served on a later turn.
Invalid choice explained and asked again	stage1-victory turns 1 and 3 ("abc", serving with no READY order, cooking a READY order).
VIP 10% off; Critic 12 loss	stage2-victory: C3 pays $16.20 for Pasta; stage2-defeat: Critic leaves on turn 12.
ComboMeal price and units	Salad + Pasta = $24.30, 3 units (menu in the stage-3 transcripts).
VIP then Happy Hour	stage3-victory turn 7: $24.30 → $21.87 → $17.50. (Burger: $15 × 0.9 × 0.8 = $10.80.)
Happy Hour recovery and duration	stage3-defeat turns 1-2: C1 recovers 15, display shows "ON, 1 turn(s) left".
C6 cannot pay within 18 turns	All runs: C6 orders on turn 17-18 and does not pay.

6.3 Transcript excerpt (stage3-victory, turn 7)

```
Start Happy Hour now? Lasts this turn and the next (y/n): y
 > Happy Hour is ON: 20% off orders taken this turn and next turn.
 > Happy Hour: C3 (VIP) recovers to satisfaction 100.
Host: seat which queued customer? (number, 0 = wait): 3
 > Host seats C3 (VIP) at Table 1.
Waiter task (1 seat, 2 take order, 3 serve, 0 wait): 2
```

```
Which customer? (number, 0 = back): 3
 > C3 (VIP) orders Salad + Pasta Combo for $17.50.
Chef: prepare whose order? (number, 0 = wait): 3
 > Chef prepares C3's Salad + Pasta Combo (1/3).
```

7. Development process and change analysis

Transition	New files	Changed files and why
Stage 1 → 2	VIPCustomer, CriticCustomer	`Customer`: new `applyDiscount` hook. `Waiter`: uses it when the order is taken. `Menu`: `getDishWithMostPrepUnits` for the Critic. `GameController`: arrival line-up only. `ConsoleUI`: shows loss per turn. The turn steps did not change.
Stage 2 → 3	Host, ComboMeal, HappyHour	`Menu`: holds combos. `Customer`: combo rule and `recover`. `Waiter`: applies Happy Hour after the customer discount. `Restaurant`: owns HappyHour, recovers waiting customers. `GameController`: two new steps. `ConsoleUI`: new prompts and status. `Order`, `Chef`, `Cashier`, `Table` unchanged.

Compare links: `.../compare/stage-1...stage-2` and `.../compare/stage-2...stage-3` (full URLs in the README).

8. Reflections

- Planning the Customer class as abstract in Stage 1 made Stage 2 easy: VIP and Critic only override methods, and the controller did not change.
- Making ComboMeal a kind of MenuItem meant the kitchen and payment code worked for combos straight away.
- Keeping all rules out of `ConsoleUI` made the game easy to replay from scripts, which we used as our test notes.
- If we had more time, we would add a UI interface so the controller could be tested without the console, and split `Restaurant` into smaller classes.
- [Each member: add a personal reflection here.]